© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_2

2. First Contact: “Hello, LLM” with Spring Boot

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

In this chapter, you will learn the basic concepts associated with LLMs from a practical viewpoint using Java.

Introduction

In this chapter, you will get your hands dirty with some coding. What fun is there without writing code? You will learn as you go throughout this chapter many interesting facts about large language modesl (LLMs) and their ecosystem. First, you will learn about OpenAI and its importance in LLM development. Then you will start exploring some of its application programming interfaces (APIs) through a Spring Boot application. We will not limit the discussion to just OpenAI but try out different LLMs. This is similar to how you would try different databases to see which one best suits you. Just remember one thing that will help you in the world of generative AI (Gen AI): there is change happening every day, so you need to keep exploring and learning.

What Is OpenAI?

OpenAI was founded in December 2015 by visionaries including Sam Altman, Greg Brockman, Ilya Sutskever, and Elon Musk with an ambitious goal of ensuring that artificial general intelligence (AGI) benefits all of humanity. Originally founded as a charity, OpenAI sought to progress artificial intelligence in a manner consistent with human ideals and needs.

Particularly in the field of generative artificial intelligence, OpenAI’s path has been distinguished by innovative artificial intelligence developments. One pieceof evidence of OpenAI’s creative energy is the Generative Pre-trained Transformer (GPT) series’ growth. Beginning with GPT-1 and working through GPT-4, every iteration has shown amazing gains in knowledge and creation of human-like prose. These models have created new benchmarks in natural language processing (NLP), allowing uses ranging from content production to conversational agents.

OpenAI debuted GPT-4o, a multimodal model able of analyzing and creating text, pictures, and audio, in May 2024. In many benchmarks, GPT-4o produced state-of-the-art performance, setting new benchmarks in audio speech recognition and translating. On the Mass Multitask Language Understanding (MMLU) test, it achieved 88.7%, higher than its predecessor, GPT-4.

OpenAI’s impact transcends language models. The company has created DALL-E, an artificial intelligence system that generates graphics from textual descriptions, therefore highlighting the possibilities of generative artificial intelligence in visual creativity. OpenAI also unveiled Codex, a technology meant to convert natural language into code that drives GitHub Copilot to help programmers with coding chores.

OpenAI's progress has been much aided by cooperation. Particularly important is the cooperation with Microsoft, which incorporates OpenAI's ideas into Microsoft's goods and services including the Azure OpenAI Service. This cooperation has helped several sectors implement AI technologies, hence improving production and creativity.

OpenAI's programs to guarantee safety and democratize AI access clearly show its dedication to ethical AI development. The company has supported studies on artificial intelligence morality and participated in debates on the responsible application of AI technologies.

By means of its pioneering research, creative products, and strategic alliances, Open AI keeps a major influence on the direction of artificial intelligence, ensuring that its developments are safe, ethical, and good for all.

Creating Your First Spring Boot LLM Application

OpenAI has created many small and large language models that are proprietary in nature. These models are hosted by OpenAI internally and exposed to the world through software development kits (SDKs) and APIs. In the current context, OpenAI does not provide an SDK in the Java Language, although it supports in Python. Python is the predominant SDK that is supported across the ecosystem. This should not be a reason for us to be disheartened, though, as we will see in later chapters the increasing plethora of ecosystems in development and use in the Java environment.

There are two ways to use LLMs. You can use an API for an LLM hosted by another system or host your own LLM in a server and expose it through an API. We will cover both of these uses in this book. In this chapter, you will try the first method, which uses the API exposed by OpenAI to interact with the LLM.

Figure 2-1 shows the different components that you will learn to build and leverage to integrate the OpenAI API with a Spring Boot application.

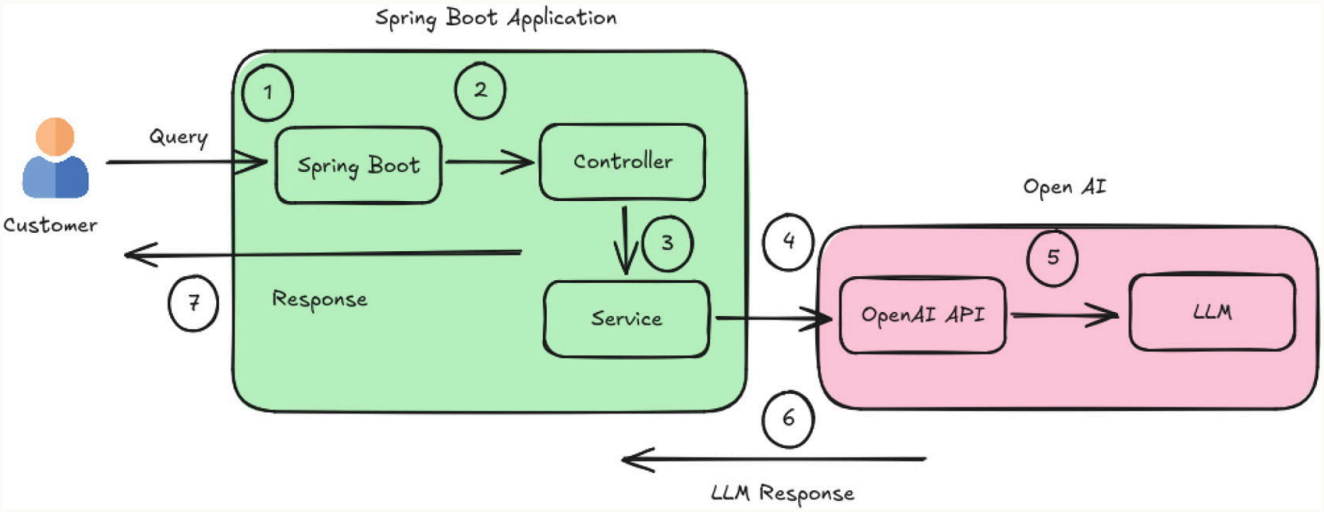


Figure 2-1 Request flow through Spring Boot application to Open AI API

First head to the Spring Initializr site at https://start.spring.io/ and create a starter template for the project with a Spring Web dependency, as shown in Figure 2-2.

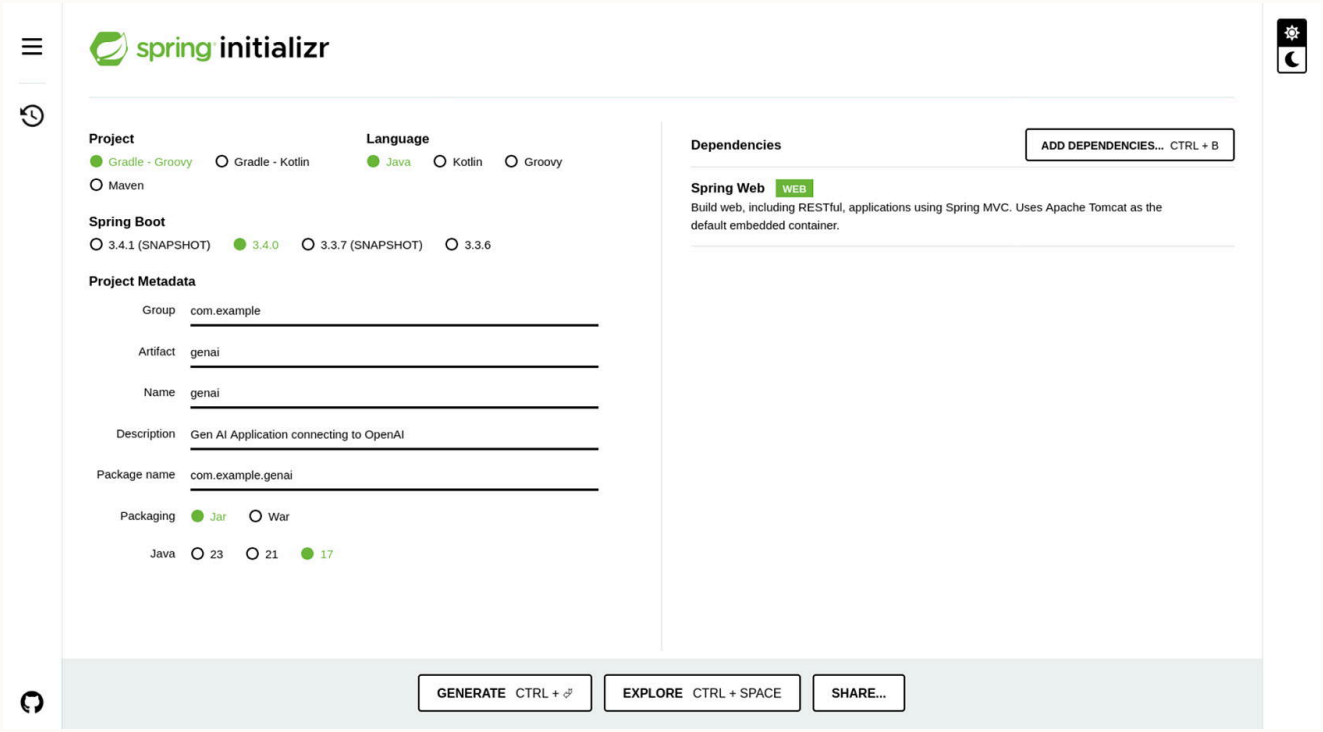


Figure 2-2 Spring Initializr website with Spring Web dependency for basic application setup

These are the prerequisites:

- Java 17+
- Open AI API Key
- IntelliJ or VSCode IDE
- Spring Boot Knowledge

Let's unzip the downloaded template, open it in our IDE of choice, and run it, as shown in Figure 2-3.

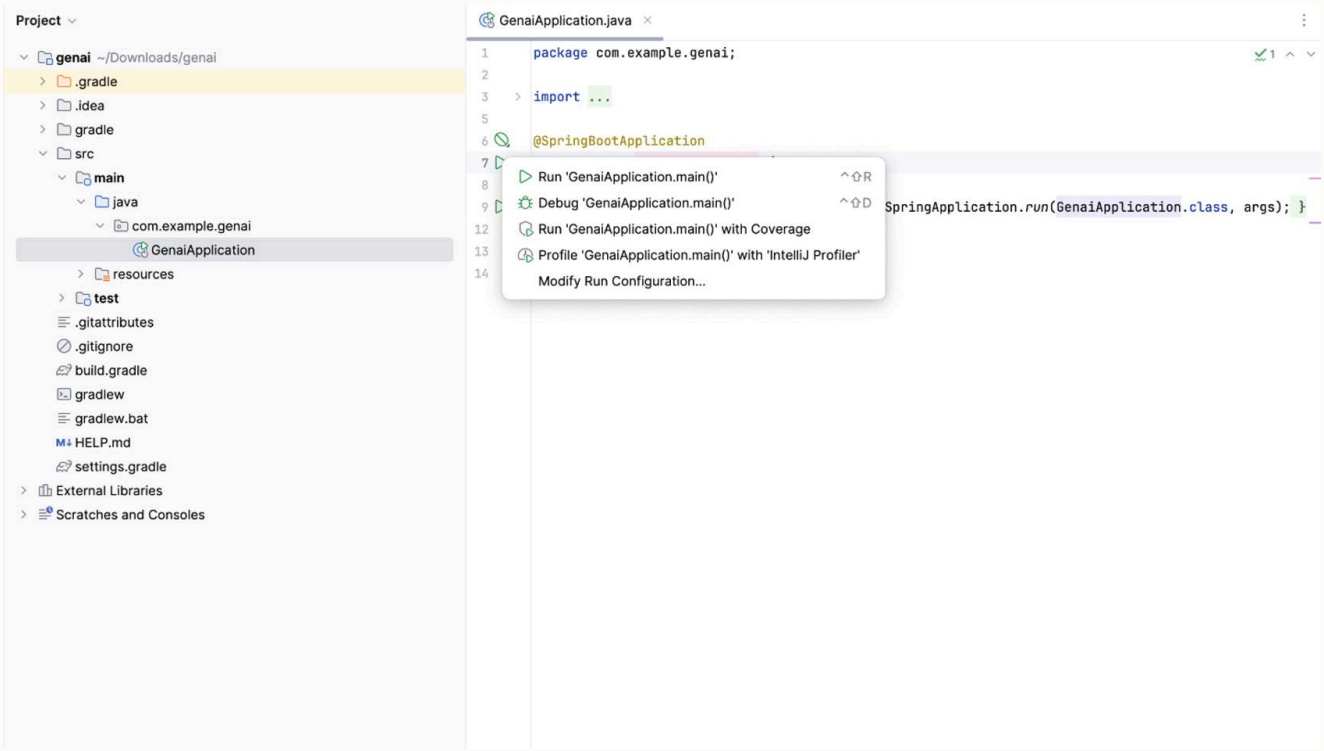


Figure 2-3 Starting the Spring Boot application from the IntelliJ IDE

Let's head over to http://localhost:8080 to see our application running and also verify it in the console. Figure 2-4 shows the Spring boot application running, and Figure 2-5 shows the response when accessing the application at http://localhost:8080. This means your application has started correctly.

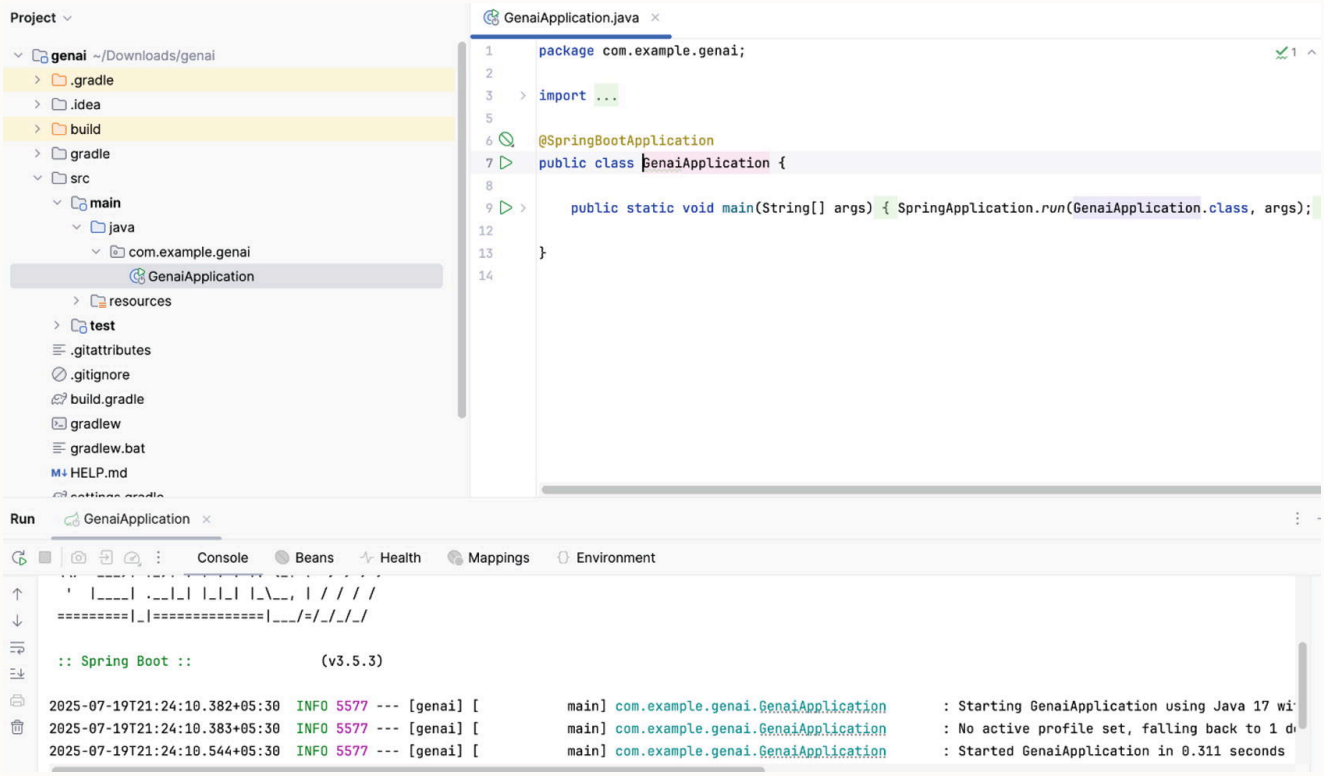


Figure 2-4 Spring Boot startup success



Figure 2-5 Spring Boot API response at http://localhost:8080

Exploring the OpenAI Chat API

This verifies your application is bootstrapped, so you are good to start from here. OpenAI provides a RESTful API to interact with its hosted LLMs. The API documentation and list can be referenced by visiting the page at https://platform.openai.com/docs/api-reference/introduction (Figure 2-6).

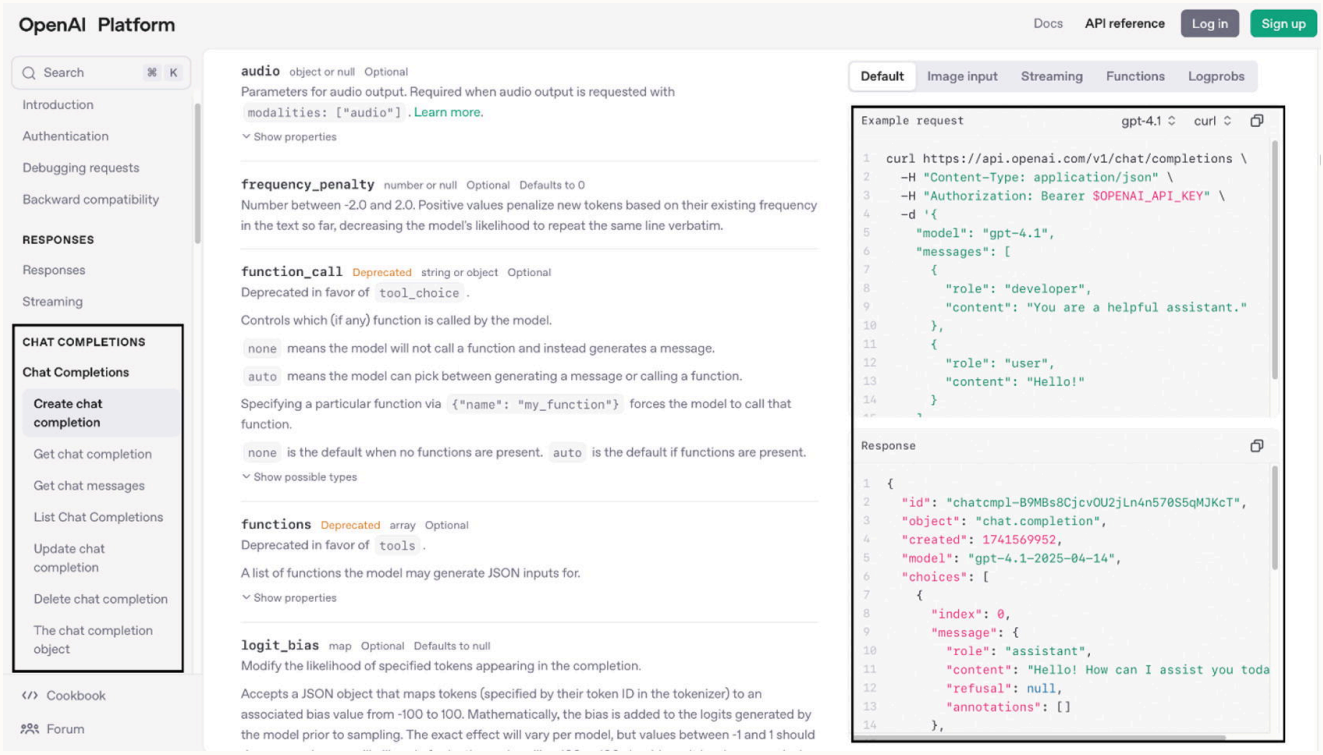


Figure 2-6 OpenAI API reference

You can leverage the APIs available in the sidebar panel displayed as endpoints, particularly starting with Chat (see Figure 2-7).

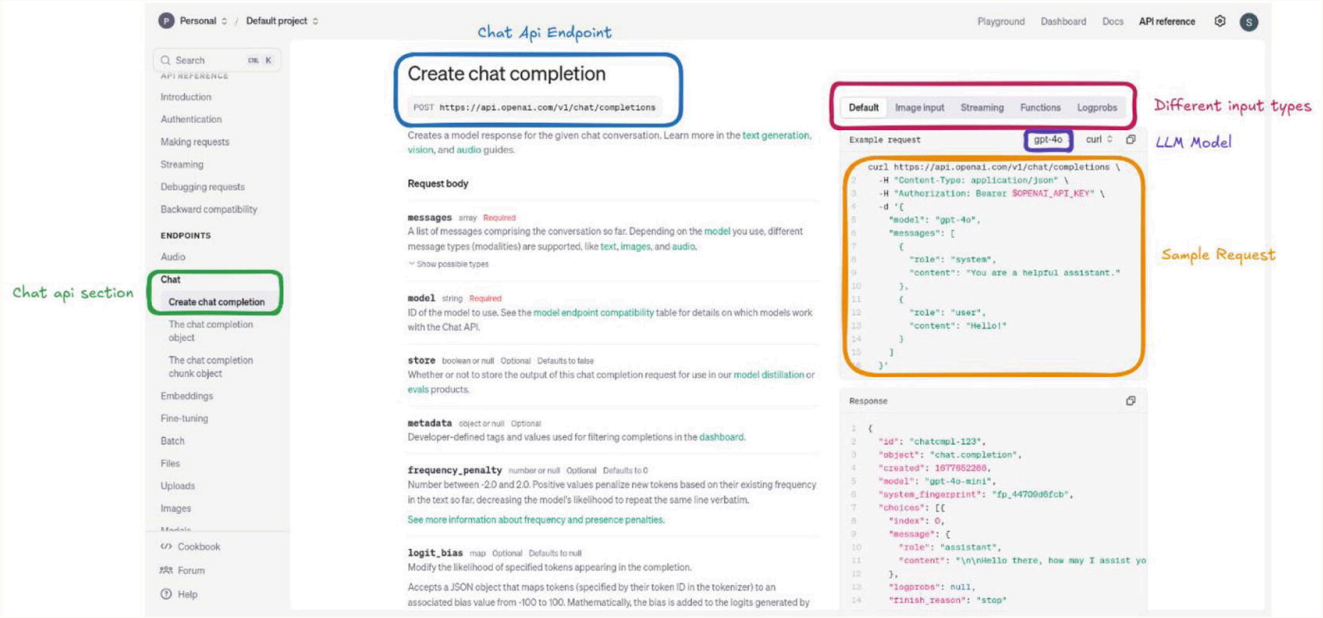


Figure 2-7 OpenAI API reference for Chat

Go to the Chat API section, as highlighted in Figure 2-7. Once in this section, you can see the documentation related to the endpoint and request body parameters in the central section. Also, it has a handy interface to choose the different input format types such as text, image, and corresponding curl request. In the same interface, you can choose from the different LLM models supported by OpenAI.

If you observe the curl request carefully, you can see it needs an OPENAI_API_KEY that can be created from the dashboard page at https://platform.openai.com/api-keys, as shown in Figure 2-8.

```
curl https://api.openai.com/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $OPENAI_API_KEY" \
```

```
 -d '{
"model": "gpt-4o-mini",
"messages": [
  {
    "role": "system",
    "content": "You are a helpful assistant."
  },
  {
    "role": "user",
    "content": "Hello!"
  }
]
 }'
```

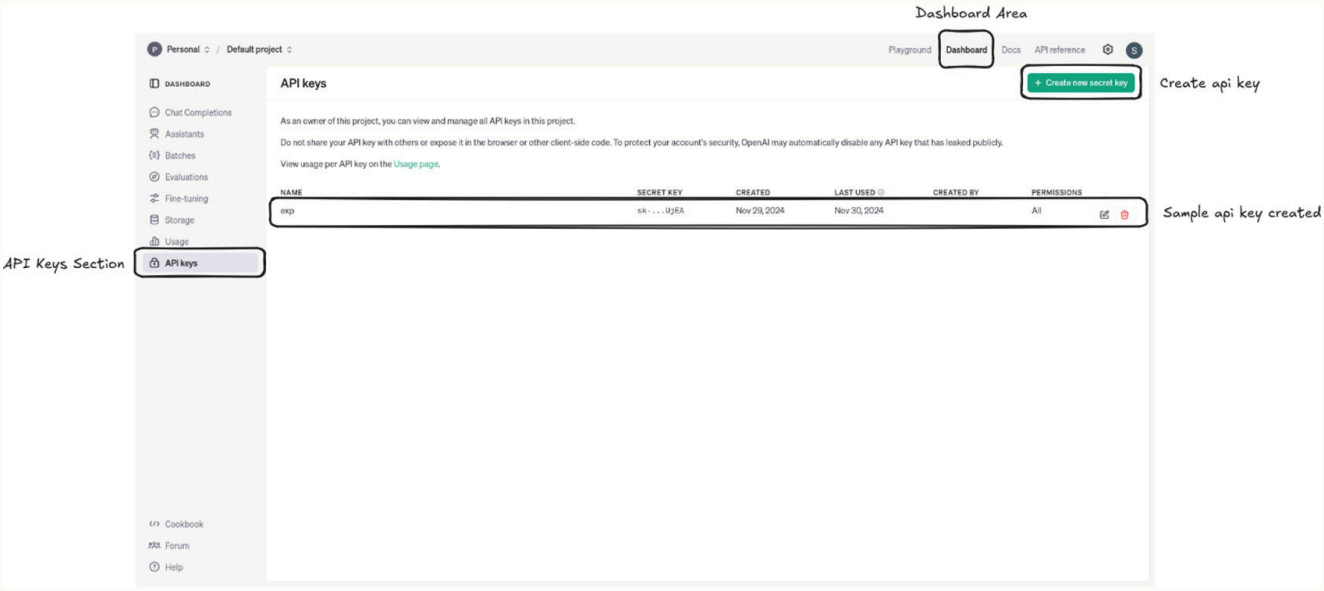


Figure 2-8 OpenAI API secret key creation

Visit the dashboard page by clicking at the top, visiting the API keys section, and clicking the "Create new secret key" button, as highlighted in Figure 2-8. To create an OPENAI_API_KEY, you will need a paid account. It costs a few bucks but is worth the cost in terms of the future applications that you are going to create that leverage LLMs.

Now that you have an OpenAI API key, let's execute a sample curl request to get a "Hello!" message from it. First export your key into the environment so that you don't have to write it directly into your curl command for the best safety. Using the following curl request, you are suggesting to the LLM that it should act as a helpful assistant. This tells it the context of the interaction it is supposed to be in. The following message "Hello!" is our first message to it in this context, and we want to know how it will respond to us.

```
$ export OPENAI_API_KEY=YOUR_API_KEY
$ curl https://api.openai.com/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $OPENAI_API_KEY" \
 -d '{
"model": "gpt-4o-mini",
"messages": [
  {
    "role": "system",
    "content": "You are a helpful assistant."
  },
  {
```

```
    "role": "user",
    "content": "Hello!"
  }
]
}'
```

Output:

```
{
 "id": "chatcmpl-AZcBXXqaydyddov2mtXXBXtV0b9fK",
 "object": "chat.completion",
 "created": 1733051627,
 "model": "gpt-4o-mini-2024-07-18",
 "choices": [
  {
   "index": 0,
   "message": {
    "role": "assistant",
    "content": "Hello! How can I assist you today?",
    "refusal": null
   },
   "logprobs": null,
   "finish_reason": "stop"
  }
 ],
 "usage": {
  "prompt_tokens": 19,
  "completion_tokens": 9,
  "total_tokens": 28,
  "prompt_tokens_details": {
   "cached_tokens": 0,
   "audio_tokens": 0
  },
  "completion_tokens_details": {
   "reasoning_tokens": 0,
   "audio_tokens": 0,
   "accepted_prediction_tokens": 0,
   "rejected_prediction_tokens": 0
  }
 },
 "system_fingerprint": "fp_0705bf87c0"
}
```

Congratulations on your first interaction with an LLM! You can see in the output section in the JSON structure a field called *choices* that is a list of outputs. The first output is the message from the LLM in the subfield called *content*. If you look carefully, the output is also a field called *usage*, which displays a few fields called prompt_tokens, completion_tokens, and total_tokens. To understand this, you can compare it to the CPU limit for software applications.

Specifically, every software application has an optimal CPU requirement to perform and will not work if the CPU processing power is less than that. And if you are in a cloud environment, you are charged on the CPU type usage based on hours.

Similarly, tokens are the basic unit of messaging in the LLM world. Each LLM has a maximum token limit it can handle per request, and OpenAI charges based on the total number of tokens used. prompt_tokens is the number of tokens in our input message called the *prompt message*, and completion_tokens is the output message from the LLM. Both together are the total tokens used by the LLM to respond. You will learn more about this terminology later in the chapter, so do not worry. For now, you will learn as you go.

Integrating OpenAI API with Spring Boot

Now that you have your API working, the next task is to integrate it with a Spring Boot application. You will first create a controller with a GET request type route so that you have an API that can be exposed to your end users. Then you will create a service that will connect and request to OpenAI API and finally connect the controller to use the service.

The following is the final code structure, and Figure 2-9 shows the component use and interaction.

```
main
├── java
│   └── com
│       └── example
│           └── genai
│               ├── GenaiApplication.java
│               ├── controller
│               │   └── ChatController.java
│               ├── model
│               │   ├── Message.java
│               │   ├── Choice.java
│               │   ├── ChatCompletionRequest.java
│               │   ├── ChatCompletionResponse.java
│               │   └── ChatRequest.java
│               └── service
│                   └── ChatService.java
└── resources
    ├── application.properties
    ├── static
    └── templates
```

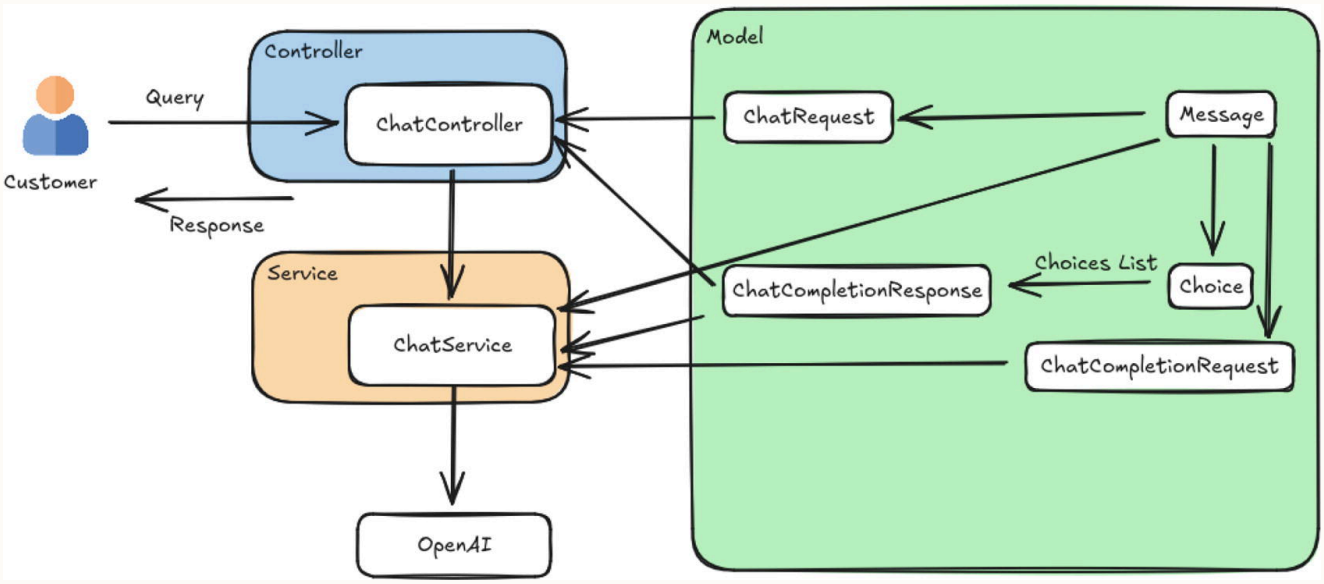


Figure 2-9 Spring Boot basic components for OpenAI API integration

Let's start by focusing on the model POJO entities first. From the curl request and response, you see a particular structure around which you can create four POJO objects: Message, Choice, ChatCompletionRequest, and ChatCompletionResponse. These will be used by the ChatService class to map the message format, request, and response.

So first, create a directory called model and create three files with the following content:

File: model/Message.java

```
package com.example.genai.model;
public class Message {
    private String role;
    private String content;
   public String getRole() {
        return role;
    }
    public void setRole(String role) {
        this.role = role;
    }
    public String getContent() {
        return content;
    }
    public void setContent(String content) {
        this.content = content;
    }
}
```

In the previous file, you are creating getter and setter attributes and methods for the Message entity for the role and content of the message, as shown in the following response element:

```
$ curl https://api.openai.com/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $OPENAI_API_KEY" \
 -d '{
"model": "gpt-4o-mini",
"messages": [
  {
    "role": "system",
    "content": "You are a helpful assistant." <-- Message POJO Object
  },
  {
    "role": "user",
    "content": "Hello!"
  }
]
 }'
```

File: model/ChatCompletionRequest.java

```
package com.example.genai.model;
import java.util.List;
public class ChatCompletionRequest {
    private String model;
```

```
    private List<Message> messages;
    public String getModel() {
        return model;
    }
    public void setModel(String model) {
        this.model = model;
    }
    public List<Message> getMessages() {
        return messages;
    }
    public void setMessages(List<Message> messages) {
        this.messages = messages;
    }
}
```

The Message entity acts as the unit message object, which then needs to be collected and sent in the messages field along with the model parameter. This is contextualized in the ChatCompletionRequest.java file through getters and setters for the model and messages parameters. The relevant sections for these in the curl request are shown here:

```
$ curl https://api.openai.com/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $OPENAI_API_KEY" \
 -d '{
"model": "gpt-4o-mini", <-- ChatCompletionRequest POJO Object
"messages": [                                ^
  {                                          |
    "role": "system",                        |
    "content": "You are a helpful assistant." <-- Message POJO Object
  },
  {
    "role": "user",
    "content": "Hello!"
  }
]
 }'
```

Now that we have our request-related POJO objects, let's focus on the response POJO object structure. Reviewing the following curl response, you will need two POJO objects, one for the inner choice and one for an outer response POJO. The other fields are not relevant for our use case at this moment, so you do not need to parse them.

```
{
  "id": "chatcmpl-AZcBXXqayd...",     <--- ChatCompletionResponse POJO
  "object": "chat.completion",                ^
  "created": 1733051627,                      |
  "model": "gpt-4o-mini-2024-07-18",          |
  "choices": [                                |
    {                                  <--- Choice POJO
      "index": 0,                              ^
      "message": {                     <--- Message POJO
      "role": "assistant",
      "content": "Hello! How can I assist you today?",
      "refusal": null
```

```
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    ...
  },
  "system_fingerprint": "fp_0705bf87c0"
}
```

Now you will create getter and setter objects for the previous POJO objects. The Message POJO is already available, so you do need not to create it again.

File: model/choice.java

```
package com.example.genai.model;
public class Choice {
    private int index;
    private Message message;
   public int getIndex() {
        return index;
    }
   public void setIndex(int index) {
        this.index = index;
    }
   public Message getMessage() {
        return message;
    }
   public void setMessage(Message message) {
        this.message = message;
    }
}
```

File: model/ChatCompletionResponse.java

```
package com.example.genai.model;
import java.util.List;
public class ChatCompletionResponse {
    private String id;
    private String object;
    private long created;
    private String model;
    private List<Choice> choices;
    public String getId() {
        return id;
    }
   public void setId(String id) {
        this.id = id;
    }
   public String getObject() {
        return object;
    }
    public void setObject(String object) {
```

```
        this.object = object;
    }
    public long getCreated() {
        return created;
    }
    public void setCreated(long created) {
        this.created = created;
    }
    public String getModel() {
        return model;
    }
    public void setModel(String model) {
        this.model = model;
    }
    public List<Choice> getChoices() {
        return choices;
    }
    public void setChoices(List<Choice> choices) {
        this.choices = choices;
    }
}
```

The base model POJO objects are all set up. They need to be composed inside the Service class next to make requests to OpenAI and fetch the response. Create a file named ChatService.java inside the service directory.

File: src/main/resources/application.properties

```
spring.application.name=genai
openai.api.key=YOUR_API_KEY  <--- Replace with your own openai api key. In production setup, use environment variables instead
```

File: service/ChatService.java

```
package com.example.genai.service;
// POJO Object References
import com.example.genai.model.ChatCompletionRequest;
import com.example.genai.model.ChatCompletionResponse;
import com.example.genai.model.Message;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import org.springframework.http.*;
import java.util.Arrays;
@Service
public class ChatService {
    @Value("${openai.api.key}")
    private String openAiApiKey;
    private final RestTemplate restTemplate;
    // Constructor injection
    public ChatService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }
    private static final String OPENAI_API_URL = "https://api.openai.com/v1/chat/completions";
    public ChatCompletionResponse getChatResponse(String userMessage) {
        // Build the request body
```

```
        ChatCompletionRequest request = new ChatCompletionRequest();
        request.setModel("gpt-4o-mini");
        Message systemMessage = new Message();
        systemMessage.setRole("system");
        systemMessage.setContent("You are a helpful assistant.");
        Message userChatMessage = new Message();
        userChatMessage.setRole("user");
        userChatMessage.setContent(userMessage);
        request.setMessages(Arrays.asList(systemMessage, userChatMessage));
        // Set up headers
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);
        headers.setBearerAuth(openAiApiKey);
        HttpEntity<ChatCompletionRequest> entity = new HttpEntity<>(request, headers);
        // Make the POST request
        ResponseEntity<ChatCompletionResponse> response = this.restTemplate.postForEntity(
                OPENAI_API_URL,
                entity,
                ChatCompletionResponse.class
        );
        return response.getBody();
    }
}
```

This completes the setup of the service layer. The controller needs to leverage the service layer to make API requests to OpenAI. For this you will need a POJO object to store the structure of the incoming JSON request from the UI side and then a controller to use this POJO object and make a request to the Service layer.

File: model/ChatRequest.java

```
package com.example.genai.model;
public class ChatRequest {
    private String userMessage;
    public String getUserMessage() {
        return userMessage;
    }
    public void setUserMessage(String userMessage) {
        this.userMessage = userMessage;
    }
}
```

File: controller/ChatController.java

```
package com.example.genai.controller;
// POJO object reference
import com.example.genai.model.ChatCompletionResponse;
import com.example.genai.model.ChatRequest;
import com.example.genai.service.ChatService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.HashMap;
import java.util.Map;
@RestController
```

```
@RequestMapping("/api/chat")
public class ChatController {
  @Autowired
   private ChatService chatService;
  @PostMapping
   public Map<String, String> chat(@RequestBody ChatRequest chatRequest) {
      ChatCompletionResponse response = chatService.getChatResponse(chatRequest.getUserMessage());
      // Extracting the content from the response
      String content = response.getChoices().get(0).getMessage().getContent();
     // Creating a response map
      Map<String, String> responseMap = new HashMap<>();
      responseMap.put("content", content);
     return responseMap;
   }
}
```

Wow! That was a whole lot of code to write. Now, to see the fruits of your hard work, let's start the application and make a curl request to see it working.

```
curl -X POST "http://localhost:8080/api/chat" -H "Content-Type: application/json" -d '{"userMessage": "Hello!"}'
{"content":"Hello! How can I assist you today?"}
```

Conclusion

In this chapter, you grasped the fundamental principles of dealing with large language models by using Java and Spring Boot. You started by exploring the relevance of OpenAI, its contributions to the evolution of innovative LLMs, and the transformative potential of tools such as GPT-4 and GPT-4o. Knowing OpenAI's development helps us to frame your practical investigation of creating an LLM-based application.

Then you created your first Spring Boot application to interact with the potent models of OpenAI. You investigated the OpenAI Chat API and discovered how to properly create API requests and manage responses. You developed several components step-by-step, starting from defining POJOs to building a service class that connects with the OpenAI API. Lastly, you combined it all in a RESTful controller to make the interaction available via a basic endpoint. The code structure lets you easily ask and get answers from the LLM, thus offering a versatile and extendable solution for next development.

You also explored several important facets of interacting with LLMs—such as tokens, usage, and how timely engineering shapes the results you obtain. This knowledge helped you to acquire a useful sense of how LLMs ingest inputs and generate outputs as well as how those map to API calls. Like CPU or bandwidth use in cloud architecture, you found that the main indicators to control LLM use are token limits and consumption.

This chapter was essentially about learning by doing—using theoretical knowledge in hands-on projects. You can now extend your abilities to work with other LLMs and have the basis required to engage properly with OpenAI's API. The next chapters will be a stepping stone derived from this pragmatic experience.

Remember that working with LLMs is always changing, much like software ecosystems are—they evolve, adapt, and grow often. As you advance, stay inquisitive, keep experimenting, and keep learning. Having these foundations in place helps you to tackle more complex ideas, hence increasing your capacity to create intelligent applications driven by LLMs. Let's keep traveling into the realm of Gen AI!

Previous chapter
1. Megabrains 101: Generative AI and LLMs Unboxed

Next chapter
3. The Transformer Saga—From Attention to Fine-Tuning